

GUÍA DE TRABAJO — EJERCICIO 2

(Control de Zonas en el Río)

1. Antes de programar: entender el problema

Debes comprender que este ejercicio combina:

- **evaluaciones lógicas** (método zonaSegura),
- **recorrido de matrices**,
- **gestión de un menú** donde el usuario puede seleccionar y consultar datos.

El error más común es intentar programar sin antes visualizar qué representa cada dato.

Preguntas clave que deben saber responder:

1. ¿Qué representa cada fila de la matriz?
→ una zona del río.
2. ¿Qué representa cada columna?
→ un atributo de esa zona:
 - columna 0 → nivel
 - columna 1 → corriente
 - columna 2 → obstáculos
3. ¿Qué condiciones determinan si una zona es segura?
Deben tenerlas claras ANTES de empezar a codificar.
4. ¿Qué acciones permitirá el menú?
 - Seleccionar un recorrido
 - Mostrarlo
 - Evaluar una zona
 - Evaluar todo el recorrido

2. Planificación del método zonaSegura

Este método es **puro**: recibe tres valores y devuelve un booleano.

Deberías:

- traducir reglas a condiciones lógicas,
- combinar condiciones con `&&`,
- identificar si alguna condición depende de otra (ej., corriente ≥ 30 implica una regla extra).

Puntos que debes reflexionar:

- ¿Es mejor guardar cada condición en variables booleanas?
→ Sí, porque luego el return final es más legible.
- ¿En qué orden aplicar las reglas?
→ Cualquiera, pero que sea comprensible.
- ¿Hay una regla especial que depende de otra?
→ Sí: si la corriente ≥ 30 , la zona debe tener 0 obstáculos.

3. Planificación del método contarZonasSeguras

Recibes una matriz. Debe recorrerla y, para cada fila, llamar a `zonaSegura`.

Conceptos que tienes que activar:

- un bucle que recorra las filas de la matriz:

```
for (int i = 0; i < recorrido.length; i++)
```
- extraer cada dato:
 - nivel = recorrido[i][0]
 - corriente = recorrido[i][1]
 - obst = recorrido[i][2]

- llamar al método anterior:
→ reutilización de código, no duplicación.
- incrementar un contador.

Preguntas de guía:

1. ¿Qué devuelve zonaSegura?
2. ¿Qué variable debe aumentar cada vez que la zona cumple todas las condiciones?
3. ¿Qué devuelve el método al final?

4. Organización del main con menú

El main debe coordinar todo.

Conceptos que debes tener claros:

- Necesitarás **una variable matriz** para almacenar el recorrido actual.
- Debes definir **los recorridos predefinidos**.
- Debes usar **un do-while** con un **switch**.

Preguntas guía:

1. ¿Qué pasa si el usuario intenta evaluar una zona antes de seleccionar un recorrido?
→ debes detectar que el recorrido es null.
2. ¿Qué pasa si el usuario elige un número de zona fuera de rango?
→ debes comprobar índices.
3. ¿Cómo mostrar una matriz por pantalla?
→ un bucle que recorra las filas.
4. ¿Por qué conviene separar todo en métodos?
→ El main solo coordina; los métodos hacen el trabajo real.

5. Relación entre los métodos y el menú

Debes visualizar la estructura:

RECORRIDO SELECCIONADO

↓

MOSTRAR ZONAS

↓

EVALUAR ZONA INDIVIDUAL → usa zonaSegura

↓

EVALUAR RECORRIDO COMPLETO → usa contarZonasSeguras

6. Errores típicos que debes evitar

- Confundir índices (usar 1 en vez de 0).
- Acceder a un recorrido que no ha sido seleccionado.
- No mostrar todos los datos de cada zona.
- Escribir las reglas todas en una línea (resta claridad).
- Intentar hacer todo en el main.

7. Resumen para los alumnos

1. Analiza las reglas lógicas de zonaSegura.
2. Recorre la matriz fila a fila en contarZonasSeguras.
3. Define los tres recorridos iniciales.
4. Mantén una variable “recorrido actual”.
5. Crea un menú robusto que gestione las opciones.
6. Nunca mezcles lógica de cálculo en el main.

GUÍA PRÁCTICA — EJERCICIO 2

1. MÉTODO: zonaSegura(int nivel, int corriente, int obstaculos)

Este método convierte reglas en condiciones booleanas.

Pista 1 — Crea variables intermedias

Este es el patrón, incompleto:

```
boolean nivelOK = (nivel >= /* ??? */ && nivel <= /* ??? */);  
boolean corrienteOK = (corriente < /* ??? */);  
boolean obstOK = (obstaculos < /* ??? */);
```

Pista 2 — Regla especial

Esta es la estructura, incompleta:

```
boolean reglaEspecial = true;  
if (corriente >= 30) {  
    reglaEspecial = ( /* condición que afecta a los obstáculos */ );  
}
```

¿Cuál es la condición?



Pista 3 — Devolver la combinación de condiciones

Esta es la forma, falta el contenido:

```
return /* condición1 */ && /* condición2 */ && /* condición3 */ &&  
/* reglaEspecial */;
```

2. MÉTODO: contarZonasSeguras(int[][] recorrido)

Debes recorrer filas y llamar al método anterior.

Pista 1 — Bucle sobre filas

Plantilla:

```
for (int i = 0; i < recorrido.length; i++) {  
    // extraer nivel, corriente, obst  
}
```

Pista 2 — Extraer cada valor

La estructura:

```
int nivel = recorrido[i][ /* columna nivel */ ];  
int corriente = recorrido[i][ /* columna corriente */ ];  
int obst = recorrido[i][ /* columna obstáculos */ ];
```

Pista 3 — Llamar a zonaSegura

¡Pista!

```
if (zonaSegura(nivel, corriente, obst)) {  
    // aumentar contador  
}
```

¿Cómo declarar e incrementar el contador?

3. MENÚ EN EL MAIN

Pista 1 — Crear las matrices de recorridos

Idea de matriz:

```
int[][] A = {
    { /* nivel */, /* corriente */, /* obst */ },
    ...
};
```

Pista 2 — Variable para guardar el recorrido elegido

Esta es la declaración necesaria:

```
int[][] recorridoActual = null;
```

Pista 3 — Estructura base del menú

Plantilla sin completar:

```
int opcion;

do {
    // imprimir menú

    opcion = sc.nextInt();

    switch (opcion) {

        case 1:
            // seleccionar recorrido
            break;

        case 2:
            // mostrar recorrido actual
            break;

        case 3:
```

```

        // evaluar zona concreta
        break;

    case 4:
        // evaluar recorrido completo
        break;

    case 0:
        System.out.println("Saliendo...");
        break;

    default:
        System.out.println("Opción no válida.");
}

} while (opcion != 0);

```

Pista 4 — Mostrar la matriz de zonas

Estructura del bucle, ¿cómo formateo el mensaje?

```

for (int i = 0; i < recorridoActual.length; i++) {
    System.out.println("Zona " + i + ": " +
        recorridoActual[i][0] + " " +
        recorridoActual[i][1] + " " +
        recorridoActual[i][2]);
}

```

Pista 5 — Evaluar una zona concreta

Estructura para rellenar:

```

int z = sc.nextInt();

if ( /* comprobar que z está dentro del rango */ ) {
    boolean segura = zonaSegura(
        recorridoActual[z][0],
        recorridoActual[z][1],
        recorridoActual[z][2]
    );
}

```



```
    );  
    // mostrar resultado  
} else {  
    // mensaje de error  
}
```

Pista 6 — Evaluar el recorrido completo

Solo dar la forma:

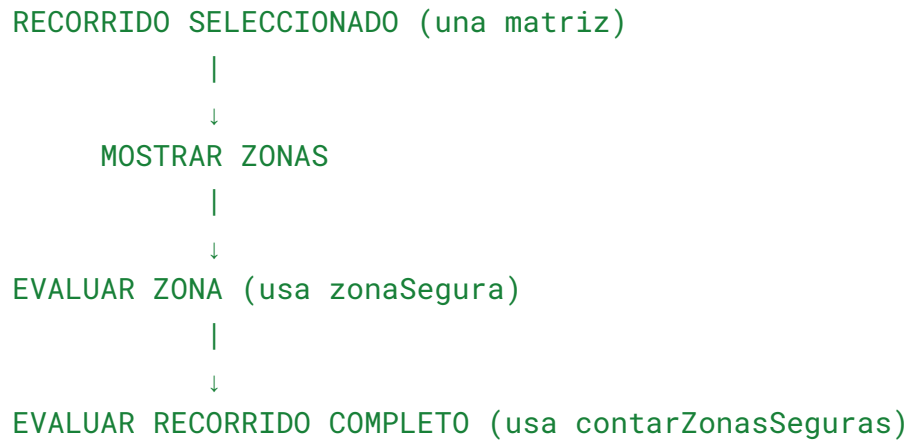
```
int seguras = contarZonasSeguras(recorridoActual);  
// mostrar seguras
```

4. Comprobaciones recomendadas

Hazte estas preguntas:

- ¿Qué pasa si el usuario intenta usar las opciones 2–4 sin haber elegido primero un recorrido?
→ necesitas un `if (recorridoActual == null)`
- ¿Qué ocurre si el usuario introduce un número de zona incorrecto?
→ índice fuera de rango → debes comprobarlo con un `if`.

5. Esquema visual para que entiendas el orden del proceso



6. Consejos que debes tener presentes

- No mezclar lógica de cálculo en el main.
- Usar bien las columnas de la matriz.
- Declarar variables claras y significativas.
- Programar cada método por separado y probarlo mentalmente.
- Comprobar siempre null antes de acceder al recorrido.